Backup Restore Testing

Day 10 | AWS / AZURE / GCP / POSTGRESQL EXAMPLE

Use this guide to investigate a specific failure, apply a cause-specific change and verify recovery. Start with evidence from the affected workload. The examples require adaptation to your environment.

How the failure happens

A backup records some recoverable state. A recovery test proves whether the required application can be reconstructed within its objectives. Missing extensions, keys, roles and infrastructure can invalidate an otherwise successful backup.

The Scenario A backup exists, but restore fails because required extensions, roles or app configuration are missing.

The Gap Backup completion does not prove a working application recovery.

The Target Measure recovery time and recovered data against RTO and RPO.

Before you start

A disposable isolated restore target; trusted backup media; required client/server compatibility; dependency inventory; application validation queries; RTO/RPO targets.

Diagnostic checks

```
# Check backup age and restore errors.
# Verify encryption-key access, extensions,
# versions, identities and target capacity.
```

Choose The Method: Managed PITR and logical pg_dump restores have different procedures.

This Example: Restore a trusted custom-format pg_dump archive into an isolated test database.

Apply the cause specific fix

Fix The Confirmed Cause

```
# Isolated lab target only; PG* auth preconfigured
createdb restore_test
pg_restore --exit-on-error \
  --no-owner --no-acl \
  --dbname=restore_test backup.dump
```

Logical Restore Example: Provision compatible PostgreSQL and required extensions first. This command writes to the test target.

Cloud-Native Restores: Also test RDS, Azure PostgreSQL or Cloud SQL recovery using that service's documented restore path.

Validate the result

```
psql -d restore_test -c "SELECT 1"

# Then run app-specific integrity checks
# and an end-to-end application transaction.
```

Not Enough: SELECT 1 proves connectivity, not complete data recovery.

Measure: Validate business records, permissions, dependencies and elapsed recovery time.

Practice exercise

Restore into a new target, reapply approved permissions, run integrity checks and execute the application flow. Record both the recovery point and elapsed recovery time.

Implementation notes

Do not run against a production database. CLI assumes a PostgreSQL client with an appropriate version and a trusted custom-format archive. --no-owner/--no-acl avoids restoring original ownership/grants; reapply tested access controls deliberately. It does not restore every cluster-level role or dependency. Managed-service snapshots/PITR are not restored with pg_restore. SELECT 1 is only a connectivity check.

Interview question

Is SELECT 1 enough to validate recovery? It proves connectivity only. Business records, application permissions and dependent services need their own checks.

Record what changed

- The symptom and affected user operation.
- Evidence supporting the root cause and alternatives ruled out.
- The changed configuration and the version or identity involved.
- Positive and negative tests, plus any remaining risks.

Example status

Examples are educational templates or fragments. They were not deployed against a live cloud account or production cluster. A successful syntax check is not proof of runtime compatibility. Test adapted configurations in an isolated environment first.

Official references

Official documentation 1